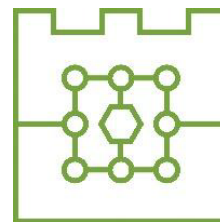




**Politechnika Krakowska
im. Tadeusza Kościuszki**

Wydział Informatyki i Matematyki



Tomasz Gondek, Łukasz Kierzek

Grupa: CY2

Kierunek: Informatyka

Specjalność: Cyberbezpieczeństwo

Raport na temat:

Analiza logów systemowych (zliczanie i filtrowanie)

Streszczenie:

Stworzenie programu mającego na celu zliczanie i filtrowanie logów systemowych wykorzystując techniki programowania równoległego.

Kraków, 2026

Spis treści

1. Cel projektu	3
2. Dane wejściowe	3
2.1. Apache Hadoop	3
2.2. Struktura logu	3
3. Opis działania programu	4
3.1. Struktura katalogów projektu	4
3.2. Opis skryptów	4
3.2.1. main.py	4
3.2.2. benchmark_report.py	4
3.2.3. openmp_speedup.py	5
3.2.4. mpi_speedup.py	5
3.2.5. cuda_speedup.py	6
3.2.6. best_comparison.py	6
3.2.7. experiments_openmp_mpi.py	7
3.2.8. config.py	7
3.2.9. report.py	7
3.3. Typowy workflow projektu	7
3.3.1. Przygotowanie danych	7
3.3.2. Pierwsze uruchomienie analizy	8
3.3.3. Zawężenie analizy	8
3.3.4. Zapis wyników	8
3.3.5. Sprawdzenie wariantów równoległych	8
3.3.6. Pomiar wydajności	9
3.3.7. Przygotowanie wyników do raportu	9
3.3.8. Porównanie najlepszych wariantów	10
3.3.9. Weryfikacja poprawności	10
3.4. Dodatkowe pliki	10
4. Wyniki	10
4.1. Sprzęt użyty w testach	10
4.2. Najczęściej występujące słowa	11
4.3. Dodatkowe statystyki	12
4.4. Porównanie CPU i GPU	13
4.5. Wyniki OpenMP	14
4.6. Wyniki MPI	15
4.7. Wyniki CUDA	15
4.8. Zbiorcze porównanie metod	16
5. Wnioski końcowe	16

1. Cel projektu

Celem projektu było opracowanie programu analizującego logi systemowe. Program umożliwia zliczanie występowania wybranych słów i fraz, filtrowanie rekordów spełniających określone kryteria oraz generowanie statystyk opisujących występowanie zdarzeń w czasie. W projekcie zastosowano techniki programowania równoległego z wykorzystaniem OpenMP, MPI oraz CUDA/OpenCL. Równoległe przetwarzanie umożliwia efektywne wykorzystanie współczesnych architektur wielordzeniowych oraz akceleratorów graficznych, zwiększając przepustowość systemu i pozwalając na analizę znacznie większych zbiorów danych w krótszym czasie.

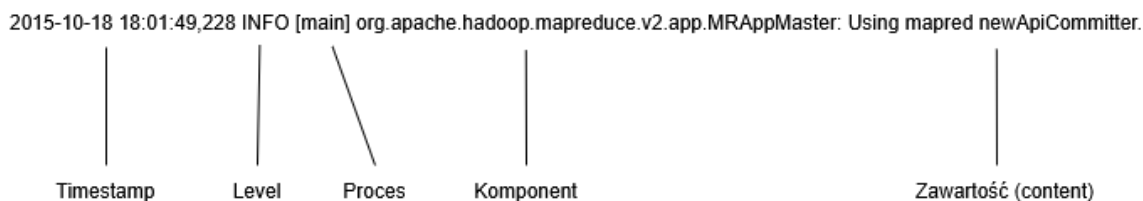
2. Dane wejściowe

2.1. Apache Hadoop

Jako dane wejściowe użyliśmy logów z Apache Hadoop, pobranych z otwartego zbioru logów <https://github.com/logpai/loghub/tree/master/Hadoop>

2.2. Struktura logu

Przykładowy, pojedynczy log z użytego zbioru ma następującą strukturę



Element	Wartość	Opis
Timestamp	2015-10-18 18:01:49,228	Data i dokładny czas wystąpienia zdarzenia wraz z milisekundami.
Level	INFO	Określa ważność komunikatu. Typowe poziomy to DEBUG, INFO, WARNING, ERROR, FATAL.
Proces	[main]	Nazwa wątku wykonującego operację. W tym przypadku główny wątek aplikacji.
Komponent	org.apache.hadoop...	Klasa lub komponent systemu, który wygenerował wpis.
Zawartość (content)	Using mapred newApiCommitter.	Właściwa informacja o wykonanej operacji lub zdarzeniu.

Tabela 1: Składnia logu

Każdy wpis w analizowanym pliku logów składa się z kilku pól zawierających informacje o zdarzeniu. Pierwszym elementem jest znacznik czasu (timestamp), określający dokładną datę i godzinę wystąpienia zdarzenia. Następnie występuje poziom logowania (log level), informujący o charakterze komunikatu, np. informacyjnym (INFO) lub błę-

dzie (ERROR). Kolejne pole zawiera nazwę wątku odpowiedzialnego za wygenerowanie wpisu, co ułatwia analizę aplikacji wielowątkowych. Następnie podawana jest nazwa klasy lub komponentu systemu będącego źródłem komunikatu. Ostatnim elementem jest właściwa treść komunikatu opisująca wykonane działanie lub zaistniałe zdarzenie. Taka struktura umożliwia skuteczne filtrowanie logów, zliczanie zdarzeń według poziomów logowania oraz tworzenie statystyk czasowych.

3. Opis działania programu

3.1. Struktura katalogów projektu

```
prir_projekt/
├─ data/
│   ├── raw/           # surowe logi Hadoop: 2k, 200k
│   └─ processed/      # przetworzone CSV
├─ domain/             # modele danych: LogRecord, AnalysisConfig,
AnalysisResult
├─ parser/             # parser linii/logów
├─ processing/         # analiza, filtry, statystyki, top słowa
├─ parallel/
│   ├── openmp/        # równoległa tokenizacja przez threads/processes
│   ├── mpi/           # podział pracy i redukcja przez mpi4py
│   └─ cuda/           # benchmark CUDA + CPU fallback
├─ visualization/     # funkcje tworzenia wykresów
├─ reports/            # wygenerowane CSV, Markdown i PNG
├─ tests/              # testy pytest
├─ *.py                # skrypty uruchamiane ręcznie
└─ *.ipynb             # notatniki eksperymentalne/prezentacyjne
```

3.2. Opis skryptów

3.2.1. main.py

Główny skrypt projektu. Służy do uruchamiania analizy logów z poziomu terminala.

Funkcje:

- wczytuje plik logów podany przez `-input`,
- parsuje linie logów,
- filtruje logi po poziomie, frazach i zakresie dat,
- liczy wystąpienia poziomów logów,
- liczy wystąpienia podanych fraz,
- liczy błędy lub inne typy zdarzeń w podziale na godziny,
- wypisuje raport w konsoli,
- opcjonalnie zapisuje raport do JSON,
- opcjonalnie zapisuje przefiltrowane linie do CSV,
- może uruchomić dodatkowy benchmark w trybie openmp, mpi albo cuda.

Przykład: `python main.py --input data/raw/Hadoop_2k.log`

3.2.2. benchmark_report.py

Skrypt generujący zbiorczy raport benchmarkowy do sprawozdania.

Funkcje:

- wczytuje plik logów i mierzy czas odczytu z dysku,
- uruchamia benchmark CPU OpenMP,
- uruchamia benchmark GPU CUDA albo CPU fallback, jeśli CUDA nie jest dostępna,
- liczy najczęstsze słowa w logach,
- porównuje przepustowość CPU i GPU,
- zapisuje wyniki do plików CSV,
- generuje wykresy PNG,
- tworzy raport Markdown

Wyniki:

- reports/benchmark_cpu_gpu.csv
- reports/top_words.csv
- reports/benchmark_report.md
- reports/plots/cpu_gpu_throughput.png
- reports/plots/top_words.png

Przykład: `python benchmark_report.py --input data/raw/Hadoop_200k.log --openmp-workers 4 --top-n 20`

3.2.3. openmp_speedup.py

Skrypt do badania przyspieszenia wariantu OpenMP-like.

Funkcje:

- uruchamia benchmark tokenizacji dla różnych liczby workerów,
- może używać backendu thread albo process,
- mierzy czas wykonania dla każdej konfiguracji,
- liczy speedup względem jednego workera,
- liczy efficiency,
- zapisuje wyniki do CSV,
- generuje wykres speedup i efficiency.

Wyniki:

- reports/benchmark_openmp.csv
- reports/plots/openmp_speedup_efficiency.png

Przykład: `python openmp_speedup.py --input data/raw/Hadoop_200k.log --threads 1 2 4 8`

3.2.4. mpi_speedup.py

Skrypt do badania przyspieszenia wariantu MPI.

Funkcje:

- uruchamia main.py w trybie MPI,
- korzysta z mpiexec, jeśli jest dostępny,
- testuje różne liczby procesów,
- mierzy czas wykonania,
- liczy speedup względem jednego procesu,

- liczy efficiency,
- zapisuje wyniki do CSV,
- generuje wykres speedup i efficiency,
- jeśli mpiexec nie jest dostępny, uruchamia tylko fallback jednoprowocowy.

Wyniki:

- reports/benchmark_mpi.csv
- reports/plots/mpi_speedup_efficiency.png

Przykład: `python mpi_speedup.py --input data/raw/Hadoop_200k.log --procs 1 2 4 8`

3.2.5. cuda_speedup.py

Skrypt do benchmarku CUDA.

Funkcje:

- uruchamia benchmark CUDA dla różnych wartości threads per block,
- mierzy czas wykonania,
- liczy liczbę tokenów,
- liczy liczbę bajtów wejściowych,
- oblicza przepustowość w GB/s,
- zapisuje wyniki do CSV,
- generuje wykres czasu wykonania i przepustowości,
- jeśli CUDA nie jest dostępna, korzysta z CPU fallbacku z modułu CUDA.

Wyniki:

- reports/benchmark_cuda.csv
- reports/plots/cuda_speedup_efficiency.png

Przykład: `python cuda_speedup.py --input data/raw/Hadoop_200k.log --threads-per-block 64 128 256 512`

3.2.6. best_comparison.py

Skrypt porównujący najlepsze wyniki różnych technologii równoległych.

Funkcje:

- uruchamia benchmark OpenMP-like,
- uruchamia benchmark MPI,
- uruchamia benchmark CUDA,
- wybiera najlepszy czas dla każdej technologii,
- tworzy tabelę porównawczą,
- zapisuje wyniki do CSV,
- generuje wykres z najlepszymi czasami wykonania.

Wyniki:

- reports/benchmark_best_comparison.csv
- reports/plots/best_comparison_openmp_mpi_cuda.png

Przykład: `python best_comparison.py --input data/raw/Hadoop_200k.log`

3.2.7. experiments_openmp_mpi.py

Stary, zastępczy entypoint projektu. Zamiast tego skryptu należy używać best_comparison.py.

3.2.8. config.py

Plik konfiguracyjny projektu.

Funkcje:

- definiuje główny katalog projektu,
- definiuje ścieżki do danych surowych i przetworzonych,
- definiuje domyślne pliki wejściowe,
- definiuje katalog raportów,
- przechowuje domyślne wartości parametrów,
- definiuje formaty dat używane w CLI.

Nie jest to skrypt uruchamiany bezpośrednio, tylko moduł pomocniczy importowany przez inne pliki.

3.2.9. report.py

Moduł odpowiedzialny za raportowanie wyników analizy.

Funkcje:

- wypisuje podsumowanie analizy w konsoli,
- wypisuje liczbę wszystkich linii,
- wypisuje liczbę poprawnie sparsowanych linii,
- wypisuje liczbę błędów parsowania,
- wypisuje zliczenia fraz,
- wypisuje zliczenia poziomów logów,
- wypisuje liczbę błędów w podziale na godziny,
- pokazuje pierwsze przefiltrowane linie,
- zapisuje raport do JSON,
- zapisuje przefiltrowane linie do CSV.

Nie jest głównym entypointem, ale korzysta z niego main.py.

3.3. Typowy workflow projektu

3.3.1. Przygotowanie danych

Na początku użytkownik wybiera plik z logami, który będzie analizowany.

Projekt korzysta z danych z katalogu:

data/raw/

Najczęściej używane są:

- Hadoop_2k.log - mały plik do szybkiego sprawdzenia działania,
- Hadoop_200k.log - większy plik do właściwych eksperymentów i benchmarków.

Wygenerowane wyniki, tabele i wykresy zapisywane są w:

reports/ reports/plots/

3.3.2. Pierwsze uruchomienie analizy

Najpierw uruchamia się analizę na małym pliku, żeby sprawdzić, czy program poprawnie czyta dane.

Przykład:

```
python main.py --input data/raw/Hadoop_2k.log
```

W tym kroku program:

- wczytuje linie logów,
- rozpoznaje ich strukturę,
- pomija linie, których nie da się poprawnie sparsować,
- zlicza poziomy logów, np. `INFO`, `WARNING`, `ERROR`,
- pokazuje liczbę błędów w podziale na godziny,
- wypisuje krótkie podsumowanie w terminalu.

3.3.3. Zawężenie analizy

Po sprawdzeniu podstawowego działania można ograniczyć analizę do konkretnych przypadków.

Przykłady:

```
python main.py --input data/raw/Hadoop_2k.log --level ERROR
```

```
python main.py --input data/raw/Hadoop_2k.log --phrase failed
```

```
python main.py --input data/raw/Hadoop_2k.log --from-date 2015-10-18 --to-date 2015-10-19
```

Dzięki temu użytkownik może sprawdzić tylko wybrane poziomy logów, znaleźć komunikaty zawierające daną frazę albo przeanalizować określony przedział czasu.

3.3.4. Zapis wyników

Jeżeli wynik analizy jest potrzebny później, można zapisać go do pliku.

Przykłady:

```
python main.py --input data/raw/Hadoop_2k.log --output-json reports/report.json
```

```
python main.py --input data/raw/Hadoop_2k.log --level ERROR --output-filtered-csv reports/filtered_error_lines.csv
```

JSON służy do zapisania pełnego podsumowania, a CSV do zapisania konkretnych linii logów spełniających warunki filtrowania.

3.3.5. Sprawdzenie wariantów równoległych

Po wykonaniu analizy sekwencyjnej można sprawdzić, jak program zachowuje się przy użyciu różnych metod równoległych.

OpenMP:

```
python main.py --input data/raw/Hadoop_2k.log --parallel-mode openmp --openmp-workers 4
```


MPI:

```
mpiexec -n 4 python main.py --input data/raw/Hadoop_2k.log --parallel-mode mpi
```

CUDA:

```
python main.py --input data/raw/Hadoop_2k.log --parallel-mode cuda --cuda-threads-per-block 256
```

W tym etapie celem nie jest już tylko analiza treści logów, ale też sprawdzenie czasu wykonania, liczby przetworzonych tokenów i przepustowości dla różnych technologii.

3.3.6. Pomiary wydajności

Po wstępnym sprawdzeniu wariantów równoległych wykonuje się dokładniejsze pomiary na większym pliku.

OpenMP:

```
python openmp_speedup.py --input data/raw/Hadoop_200k.log --threads 1 2 4 8
```

MPI:

```
python mpi_speedup.py --input data/raw/Hadoop_200k.log --procs 1 2 4 8
```

CUDA:

```
python cuda_speedup.py --input data/raw/Hadoop_200k.log --threads-per-block 64 128 256 512
```

Na tym etapie porównuje się, jak zmiana liczby workerów, procesów albo wątków CUDA wpływa na czas wykonania. Wyniki są zapisywane jako tabele CSV i wykresy.

3.3.7. Przygotowanie wyników do raportu

Po wykonaniu pomiarów generowany jest zbiorczy raport.

Przykład:

```
python benchmark_report.py --input data/raw/Hadoop_200k.log --openmp-workers 4 --top-n 20
```

Raport pokazuje:

- porównanie CPU OpenMP i CUDA,
- najczęściej występujące słowa,
- czas odczytu pliku,
- przepustowość przetwarzania,
- wykresy gotowe do użycia w sprawozdaniu.

Najważniejsze pliki wynikowe:

- reports/benchmark_cpu_gpu.csv
- reports/top_words.csv
- reports/benchmark_report.md
- reports/plots/

3.3.8. Porównanie najlepszych wariantów

Na końcu można zestawić najlepsze wyniki dla OpenMP, MPI i CUDA.

Przykład:

```
python best_comparison.py --input data/raw/Hadoop_200k.log
```

Dzięki temu łatwo sprawdzić, która metoda osiągnęła najkrótszy czas wykonania w przeprowadzonych eksperymentach.

3.3.9. Weryfikacja poprawności

Projekt można sprawdzić testami automatycznymi.

Uruchomienie:

```
pytest
```

Testy potwierdzają, że parser, filtrowanie, analiza logów, fallback CUDA i liczenie najczęstszych słów działają zgodnie z założeniami.

3.4. Dodatkowe pliki

`experiments.ipynb`

Notebook używany do eksperymentów i ręcznego sprawdzania działania projektu. Pozwala uruchamiać fragmenty analizy krok po kroku, testować różne warianty filtrowania, sprawdzać wyniki pośrednie oraz porównywać rezultaty bez konieczności każdorazowego uruchamiania skryptów z terminala.

`pipeline_presentation.ipynb`

Notebook prezentacyjny pokazujący cały przepływ pracy projektu w bardziej uporządkowanej formie. Służy do demonstracji pipeline'u: od wczytania danych, przez analizę logów i benchmarki, aż po wygenerowanie wyników oraz wykresów do raportu.

4. Wyniki

Do przeprowadzenia głównych pomiarów wykorzystano plik `Hadoop_200k.log`. Plik miał rozmiar około 15.56 MB, a podczas analizy przetworzono 1 649 983 tokeny. Czas samego odczytu pliku wyniósł około 0.023 s, co dało przepustowość odczytu na poziomie około 0.679 GB/s.

Na podstawie tych wartości można zauważyć, że w badanym przypadku samo I/O nie było głównym ograniczeniem. Większy wpływ na czas wykonania miało właściwe przetwarzanie danych, czyli tokenizacja, zliczanie oraz redukcja wyników.

4.1. Sprzęt użyty w testach

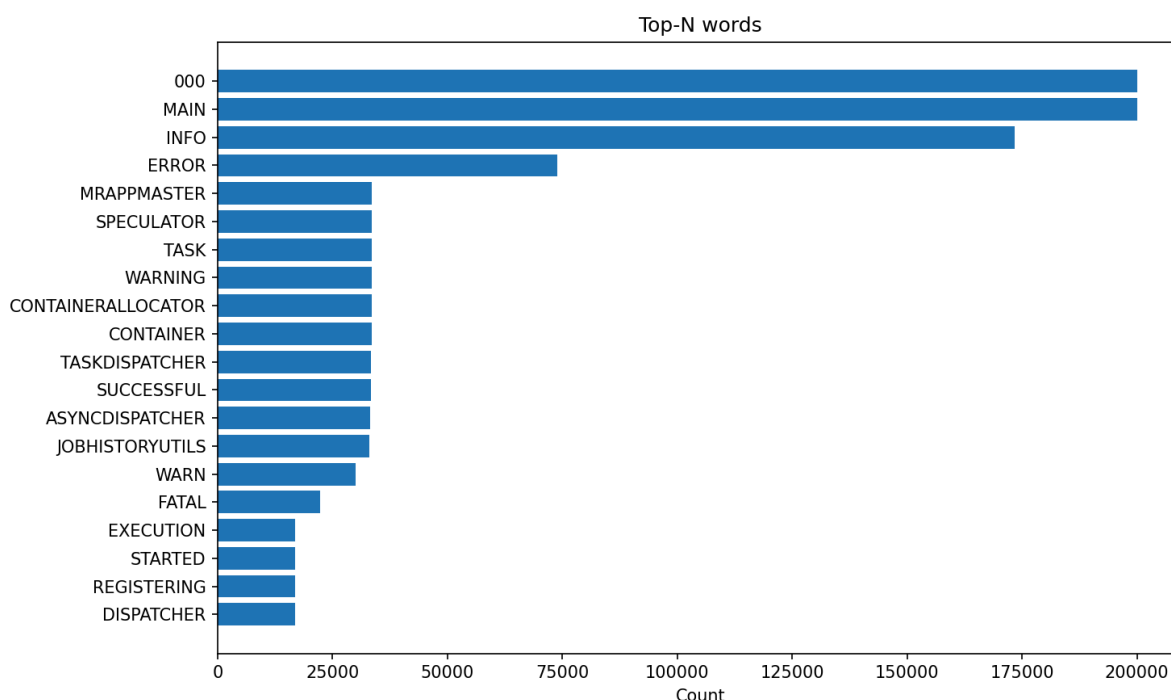
- Karta graficzna NVIDIA RTX4070
- Procesor AMD Ryzen 7 9700X
- dysk Lexar NM790 1TB M.2 2280
- Windows 11

4.2. Najczęściej występujące słowa

Jednym z wyników analizy było wyznaczenie najczęściej występujących tokenów w logach. Najwyższe wartości uzyskały tokeny **000** oraz **MAIN**, które pojawiły się po 200000 razy. Wynika to bezpośrednio ze struktury analizowanych logów, gdzie te elementy występują regularnie w wielu liniach.

Wśród najczęstszych tokenów znalazły się również poziomy zdarzeń:

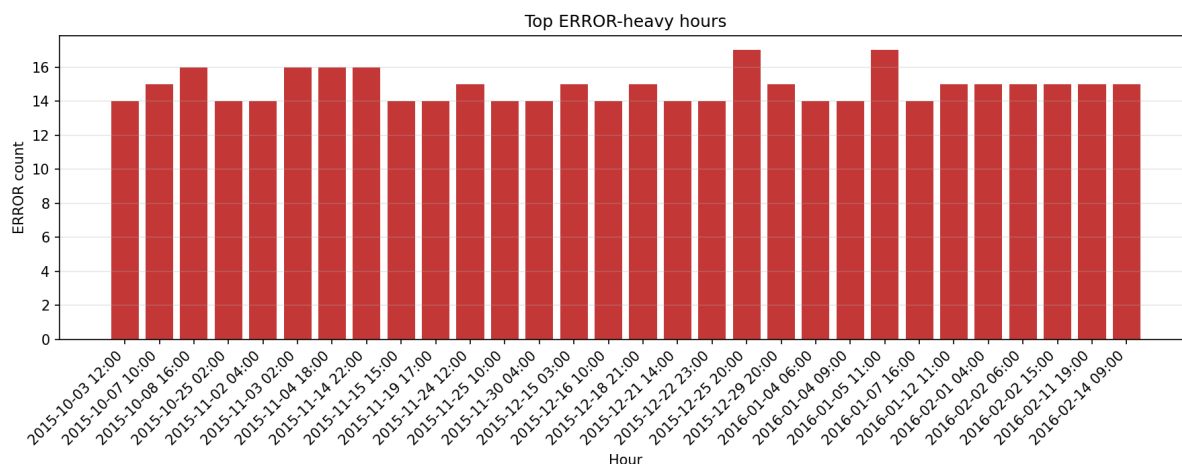
- INFO - 173322 wystąpienia,
- ERROR - 73814 wystąpień,
- WARNING - 33466 wystąpień,
- FATAL - 22285 wystąpień.



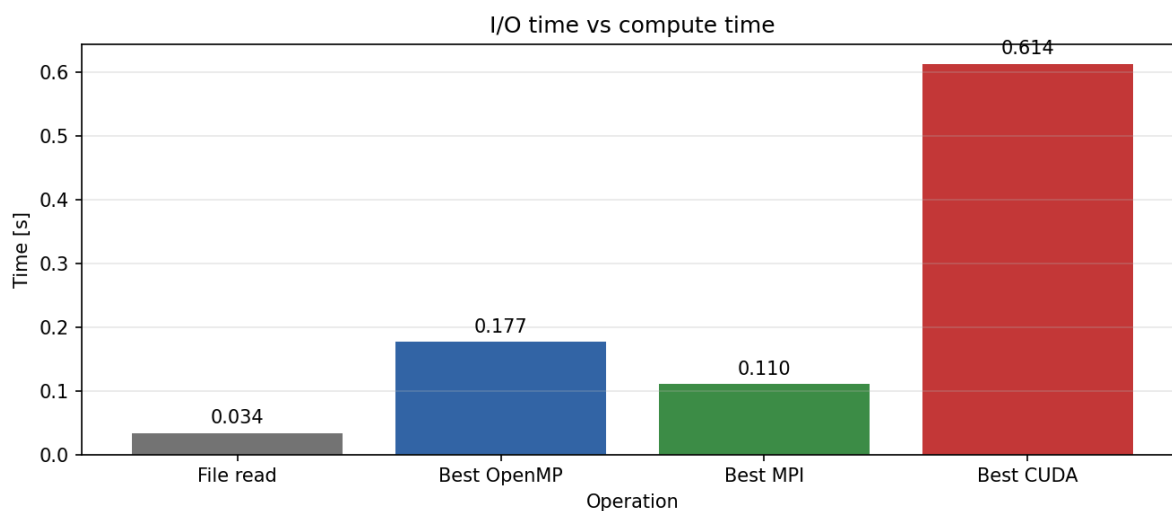
Rysunek 1: Najczęściej występujące słowa w zbiorze

Wykres pokazuje, że w danych dominują zarówno elementy techniczne powtarzające się w strukturze logów, jak i słowa opisujące typ zdarzenia. Szczególnie istotna jest duża liczba komunikatów **ERROR**, ponieważ wskazuje ona na znaczną liczbę zdarzeń błędów w analizowanym zbiorze.

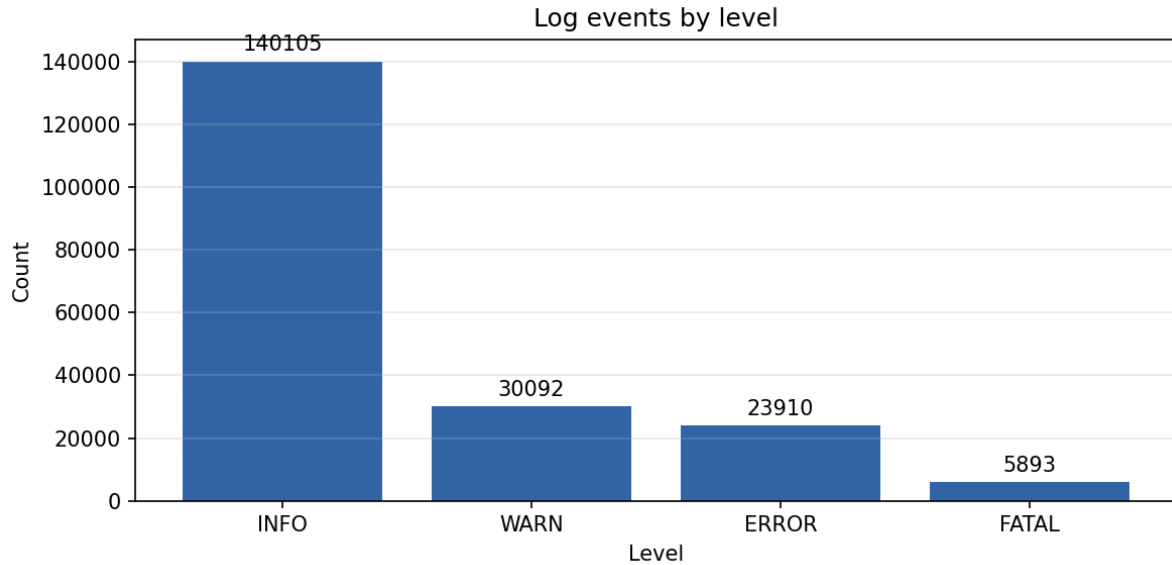
4.3. Dodatkowe statystyki



Rysunek 2: Godziny z największą ilością logów typu ERROR



Rysunek 3: Czas I/O vs czas działania



Rysunek 4: Ilość logów per dany typ (level)

4.4. Porównanie CPU i GPU

W kolejnym etapie porównano wariant CPU oparty o OpenMP z wariantem CUDA. Oba podejścia przetworzyły tę samą liczbę tokenów, czyli 1 649 983.

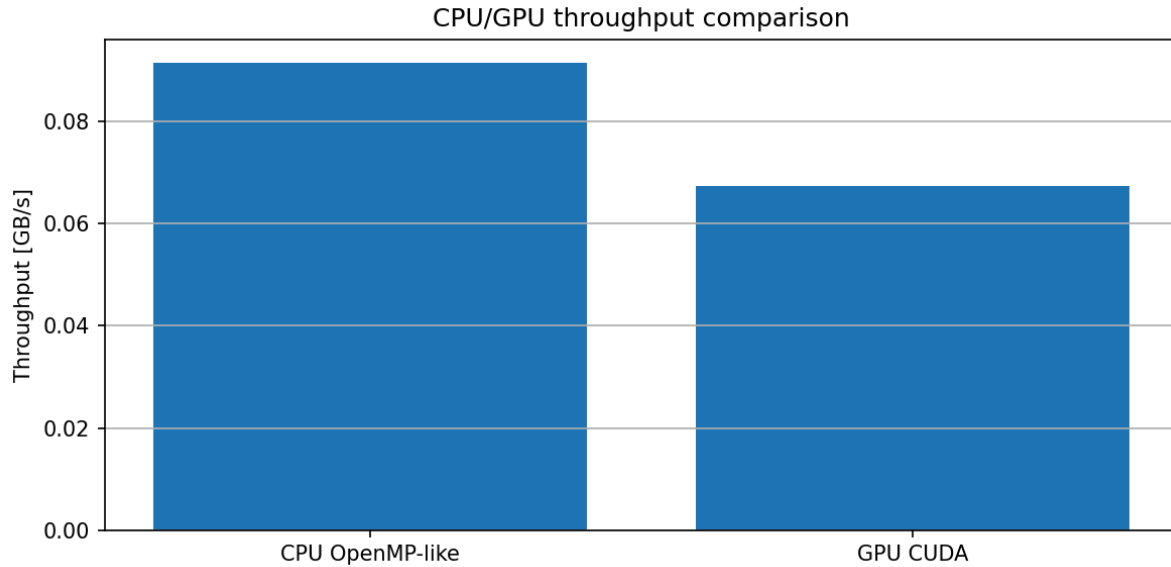
Wyniki porównania:

CPU OpenMP, 4 workery:

- czas: 0.170 s,
- przepustowość: 0.091 GB/s,
- liczba tokenów: 1 649 983.

GPU CUDA:

- czas: 0.231 s,
- przepustowość: 0.067 GB/s,
- liczba tokenów: 1 649 983.



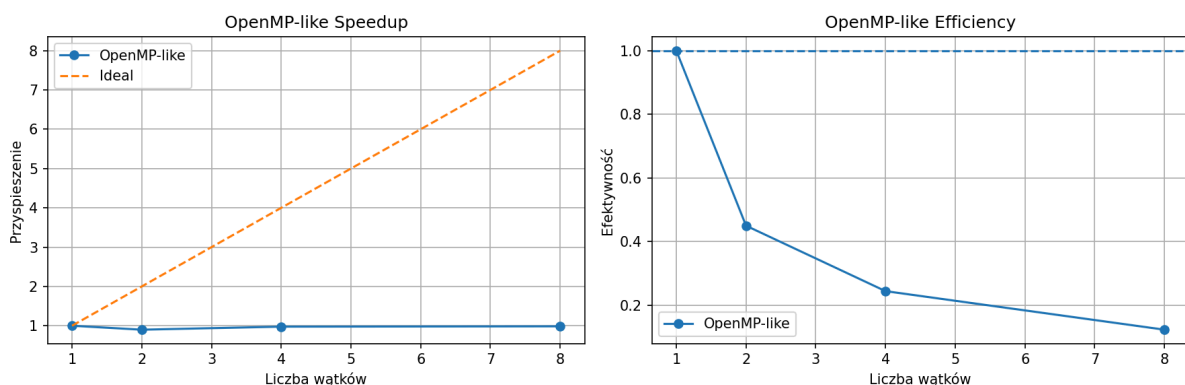
Rysunek 5: Porównanie przepustowości CPU i GPU

W tym pomiarze lepszy wynik uzyskał wariant CPU OpenMP. Czas wykonania był krótszy, a przepustowość wyższa niż w wariancie CUDA. Oznacza to, że dla badanego rozmiaru danych użycie GPU nie przyniosło przyspieszenia. Prawdopodobną przyczyną jest narzut związany z przygotowaniem danych i przenoszeniem ich na GPU, który przy pliku o rozmiarze około 15.56 MB może być większy niż zysk z równoległego wykonania na karcie graficznej.

4.5. Wyniki OpenMP

Dla wariantu OpenMP wykonano pomiary dla 1, 2, 4 i 8 workerów. Celem było sprawdzenie, czy zwiększenie liczby jednostek wykonawczych poprawia czas tokenizacji.

Otrzymane czasy były bardzo zbliżone do siebie. Dla 1 workera czas wyniósł około 0.177 s, natomiast dla 4 workerów około 0.181 s. Nie zaobserwowano więc istotnego przyspieszenia.



Rysunek 6: Przyspieszenie i efektywność OpenMP

Wykres przyspieszenia i efektywności pokazuje, że wraz ze wzrostem liczby workerów efektywność spada. Wynika to z narzutu podziału danych, uruchamiania równoległych

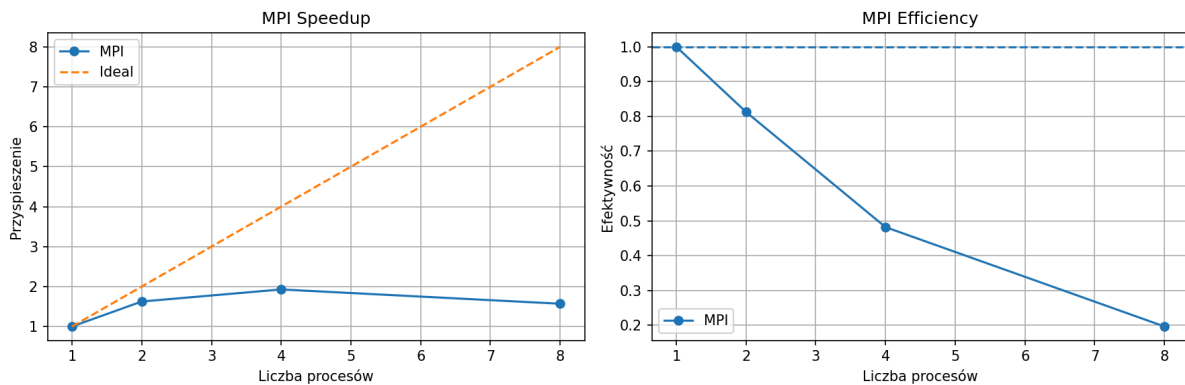
zadań oraz końcowej redukcji lokalnych słowników. W przypadku tego pomiaru koszt równoległości był większy niż uzyskany zysk obliczeniowy.

4.6. Wyniki MPI

Dla MPI przeprowadzono pomiary dla 1, 2, 4 i 8 procesów. Najlepszy wynik uzyskano dla 4 procesów.

Wyniki dla MPI:

- 1 proces: 0.213 s, speedup 1.00,
- 2 procesy: 0.131 s, speedup 1.63,
- 4 procesy: 0.110 s, speedup 1.93,
- 8 procesów: 0.135 s, speedup 1.57.



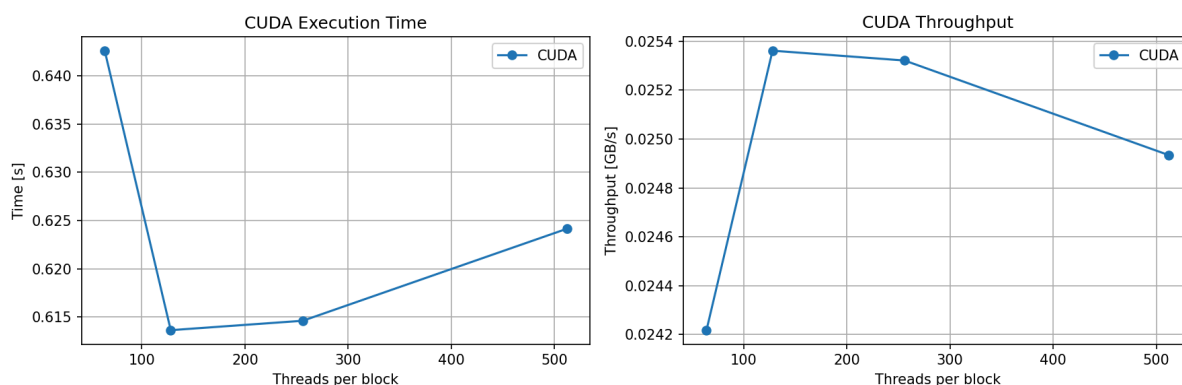
Rysunek 7: Przyspieszenie i efektywność OpenMP

Wariant MPI skalował się najlepiej spośród badanych metod. Zwiększenie liczby procesów do 4 wyraźnie skróciło czas wykonania. Dalsze zwiększenie liczby procesów do 8 pogorszyło wynik, co sugeruje, że narzut komunikacji oraz scalania wyników zaczął przewyższać korzyści z dodatkowej równoległości.

4.7. Wyniki CUDA

Dla wariantu CUDA wykonano pomiary dla różnych wartości liczby wątków w bloku: 64, 128, 256 i 512. W zapisanych wynikach pomiary zostały wykonane w trybie `cpu-fallback`, co oznacza, że właściwe obliczenia na GPU nie zostały użyte.

Otrzymane czasy mieściły się w przedziale około 0.61-0.64 s, a przepustowość wyniosła około 0.024-0.025 GB/s.

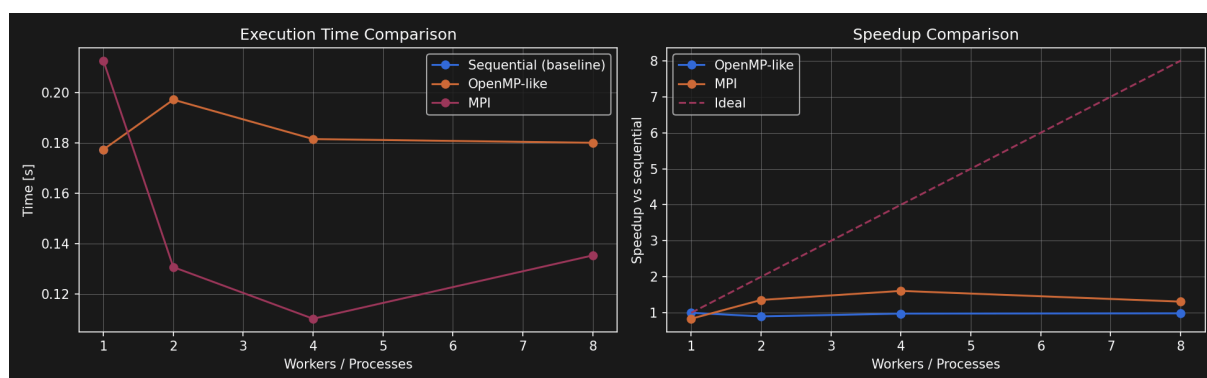


Rysunek 8: Przyspieszenie i efektywność CUDA

Ponieważ uruchomiony został fallback CPU, wyników tych nie należy traktować jako pełnego pomiaru wydajności GPU. Pokazują one jednak, że program zachowuje poprawny schemat działania także wtedy, gdy CUDA nie jest dostępna.

4.8. Zbiorcze porównanie metod

Po wykonaniu pomiarów można porównać wszystkie warianty równoległe. Najlepszy wynik uzyskano dla MPI przy 4 procesach, gdzie czas wykonania wyniósł około 0.110 s. Wariant OpenMP nie uzyskał istotnego przyspieszenia względem jednego workera, a wariant CUDA w dostępnych wynikach działał częściowo przez fallback CPU.



Rysunek 9: Porównanie OpenMP i MPI

Zestawienie pokazuje, że najlepszą metodą dla badanego przypadku było MPI. Wynika to z efektywnego podziału danych między procesy oraz stosunkowo dobrego balansu między kosztem obliczeń a kosztem komunikacji dla 4 procesów.

5. Wnioski końcowe

Uzyskane wyniki pokazują, że projekt poprawnie wygenerował zarówno statystyki zawartości logów, jak i pomiary wydajnościowe. Analiza Top-N pozwoliła wskazać najczęściej występujące tokeny, a benchmarki umożliwiły porównanie OpenMP, MPI oraz CUDA.

Najlepsze przyspieszenie uzyskano dla MPI przy 4 procesach. OpenMP nie przyniósł wyraźnej poprawy czasu wykonania, prawdopodobnie przez narzuty związane z równoległością w Pythonie.

Pomiar czasu odczytu pliku wskazuje, że I/O nie było głównym wąskim gardłem. Ograniczeniem były przede wszystkim koszty przetwarzania tekstu, podziału pracy i redukcji wyników.

Wykaz rysunków

Rysunek 1	Najczęściej występujące słowa w zbiorze	11
Rysunek 2	Godziny z największą ilością logów typu ERROR	12
Rysunek 3	Czas I/O vs czas działania	12
Rysunek 4	Ilość logów per dany typ (level)	13
Rysunek 5	Porównanie przepustowości CPU i GPU	14
Rysunek 6	Przyspieszenie i efektywność OpenMP	14
Rysunek 7	Przyspieszenie i efektywność OpenMP	15
Rysunek 8	Przyspieszenie i efektywność CUDA	16
Rysunek 9	Porównanie OpenMP i MPI	16

Wykaz tabel

Tabela 1 Składnia logu	3
------------------------------	---